CODESCRIPT – GoCode

~ The Coding Paradise

Software Engineering (IT303) Report

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

by

SANKET (201IT154)

MAYUR JINDE (201IT135)

SHAULENDRA KUMAR (201IT159)

DEPARTMENT OF INFORMATION TECHNOLOGY NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA SURATHKAL, MANGALORE – 575025

NOV, 2022

Table of Contents **Page no.**

1 Purpose…..... 2

2 Analysis….....2

3 Overview….....3

4 Features……...4

4.1 Functional Requirements … 4

4.2 Non-Functional Attributes……5

4.3 Software specifications…….5

4.3.1 Tech Stack …...…..……. 5

5 Design… 6

5.1 Activity diagram……...6

6 Data Flow Diagram…..... 7

6.1 Coding Space……...7

6.2 Problem space… 7

6.3 Contest Space……8

6.4 Personal Development Space……9

6.5 User Space…….9

6.6 Playlist space…….10

7 System Architecture ……11

8 Testing and Maintenance……..19

9 Conclusions and Future Trends……21

Purpose

The purpose of this project is to provide users with a platform where they can showcase, improve or even master their problem-solving skills. Learning Data Structures and Algorithms can be hard, but GoCode makes it convenient and fun to learn for you!!

One-stop solution for all your coding needs.

Analysis

Figure 1.

The Coding population is increasing vastly every day since everyone knows that computers are our future. There are only two major Competitive coding websites for more than a million-user base, so obviously, this particular market is yet to be utilized entirely. GoCode is built with the exact same intention of exploiting this user base. Along with the orthodox features of solving coding problems, we plan to add some cool features that we hope will attract the existing coders and beginners.

We have been involved in the coding environment ourselves and felt there are a number of problems with these websites. As a team, we feel there is much room for improvement even if not replicating the same websites would even prove profitable since the market is increasing every day and people want new tougher problems. Human mentality always wants options. They can’t stick to one particular thing when they have other options. We saw the opportunity in this market, and since we are closely related to this, we thought it is best to grab it!

Overview

There are only two major Competitive coding websites for a user base of more than a million, so obviously this particular market is yet to be completely utilized. GoCode is built with the exact same intention of exploiting this user base. Along with the orthodox features of solving coding problems, we plan on adding some cool features which we hope to attract the existing coders as well as the beginners.

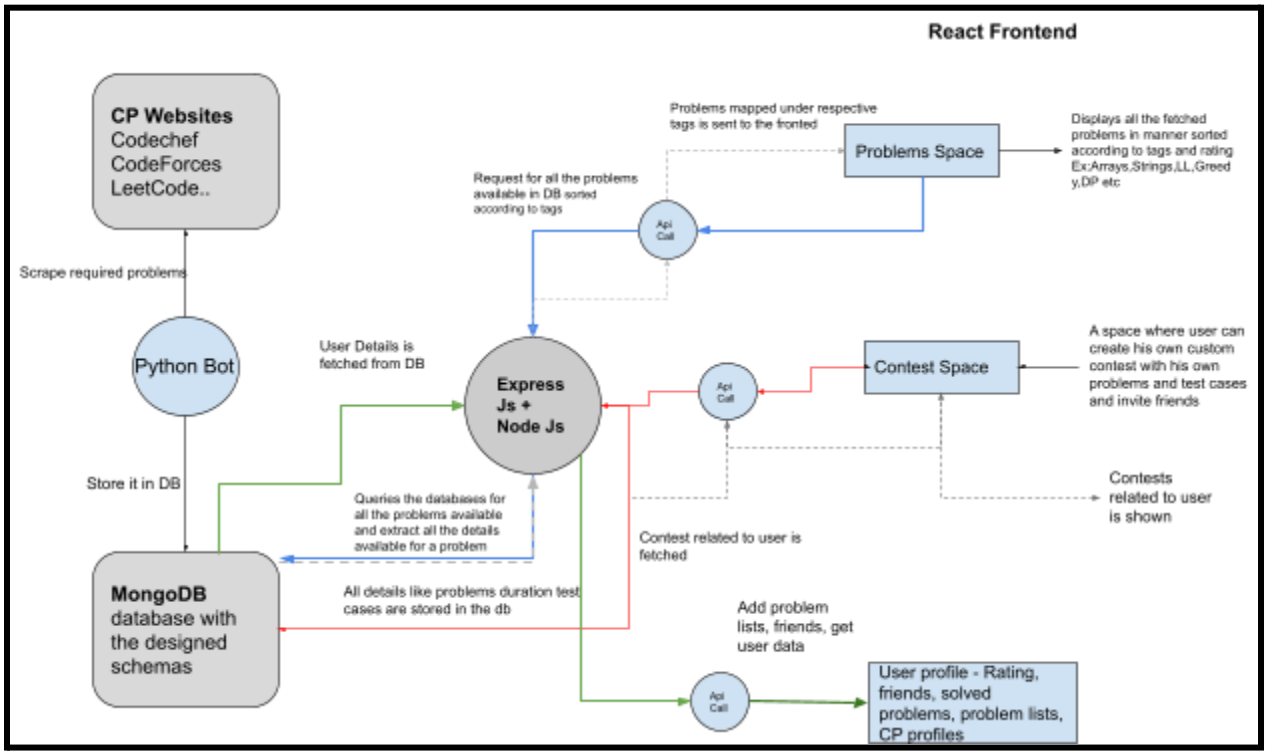

Figure 2.

Note: All the features aren't shown in the flowchart. It's just a basic idea of how we are planning to integrate various modules used.

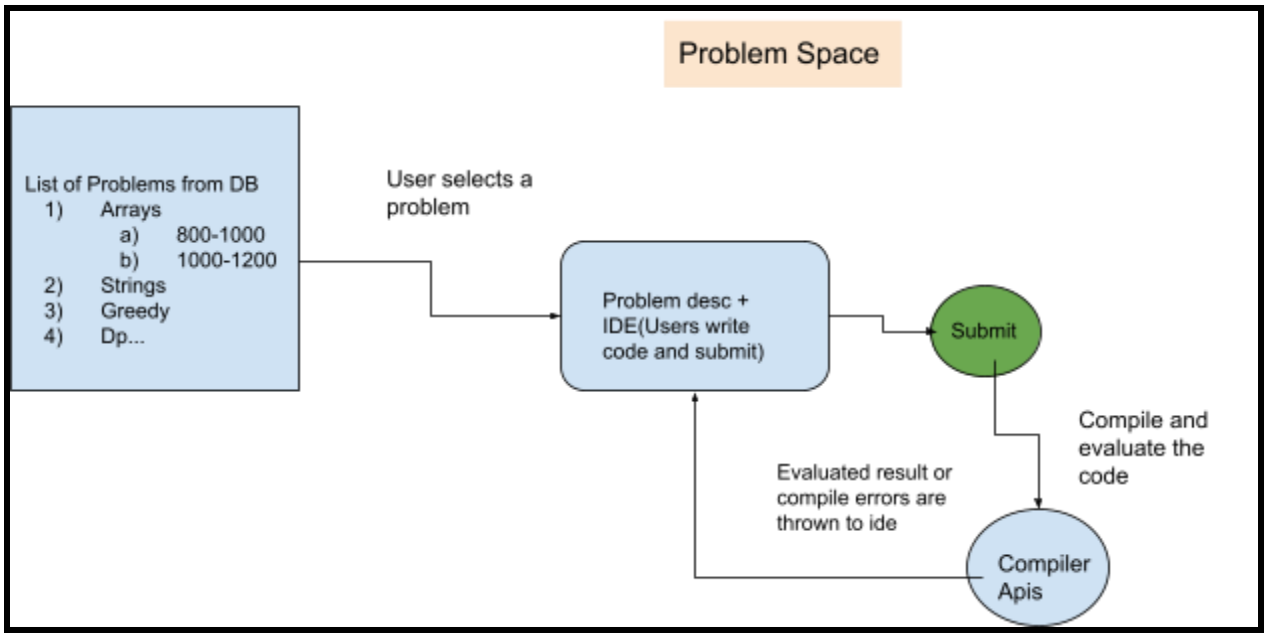

Figure 3.

Enlarged view of problem space

Features

- **Problem Space:** A plethora of problems in different categories. Each issue would be having a problem statement (that includes constraints), test cases, tags, and rating.
- **Coding Space:** Users can either use the website's basic IDE to write code and submit or upload their solution file (C, C++, Python), which will be compiled, evaluated, and corresponding results will be shown to him/her.
- **Personal Development Space:** This space is completely dedicated to a user. Here, the website recommends problems randomly based on the filters decided by the user.
- **Contest space:** Users can create their own contest with the problems of their liking and invite their friends and battle it out. Contest problems will include user-defined test cases, language options, and duration.
- **Problem list:** Users can create a list of problems of their liking and share it with others using a shareable link.
- **User Profile**: Users rating, solved problems, problem lists, friends.
- As it was mentioned by sir, our project is very transparent and we can expand it in many ways by further adding more features.

Functional Requirements:

- Scraping problems from different websites like Codeforces, HackerEarth, etc.
- Codeforces API for fetching problem lists, details, tags, and user details.
- Also, code compilation APIs for compiling the code.
- Integration of IDE for writing code.

Non-Functional Attributes:

- User authentication and authorization.
- The React frontend code can be used anywhere on the website which provides reusability.
- MongoDB is the database used. Data is protected with pre-configured security features for authentication, authorization, encryption, and more.
- React is compatible with all the available browsers.

Software specifications:

Tech Stack:

- Backend → Express.js
- Frontend → React
- Database → MongoDB
- Deployment→ Heroku/AWS/MongoDB Atlas

Design

Activity diagram

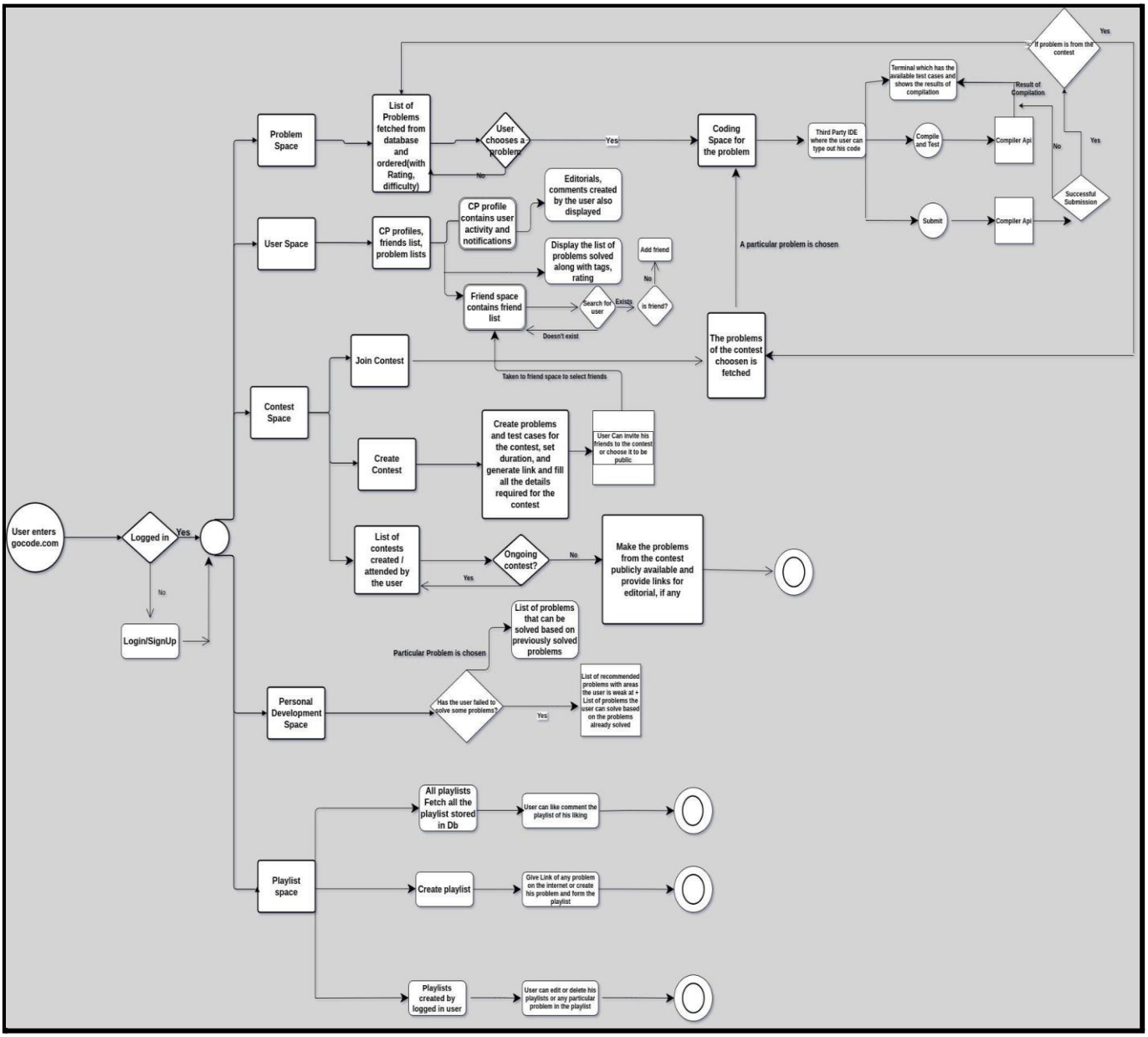

Figure 4.

Data Flow Diagram

Coding Space

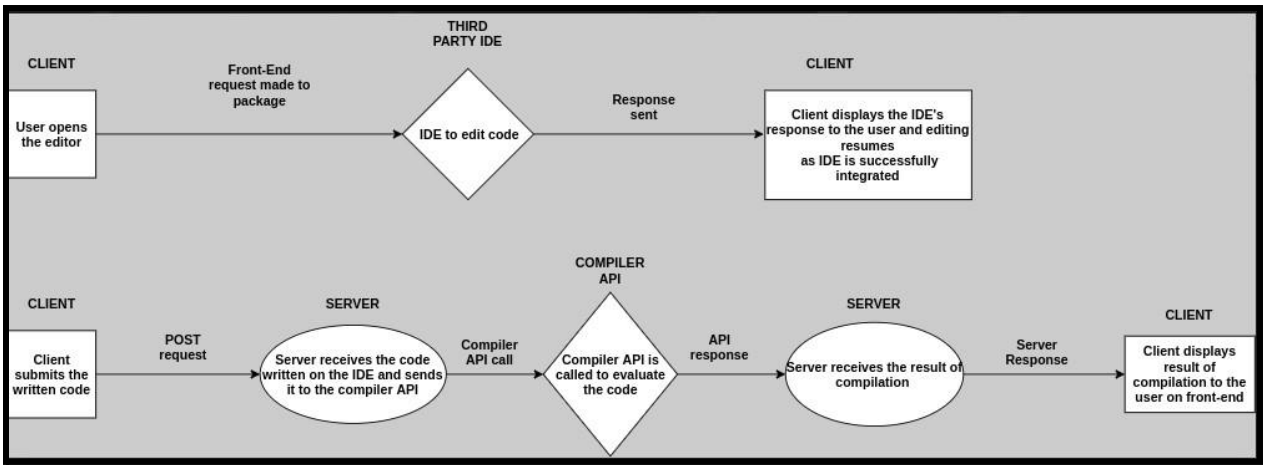

Figure 5.

Problem space

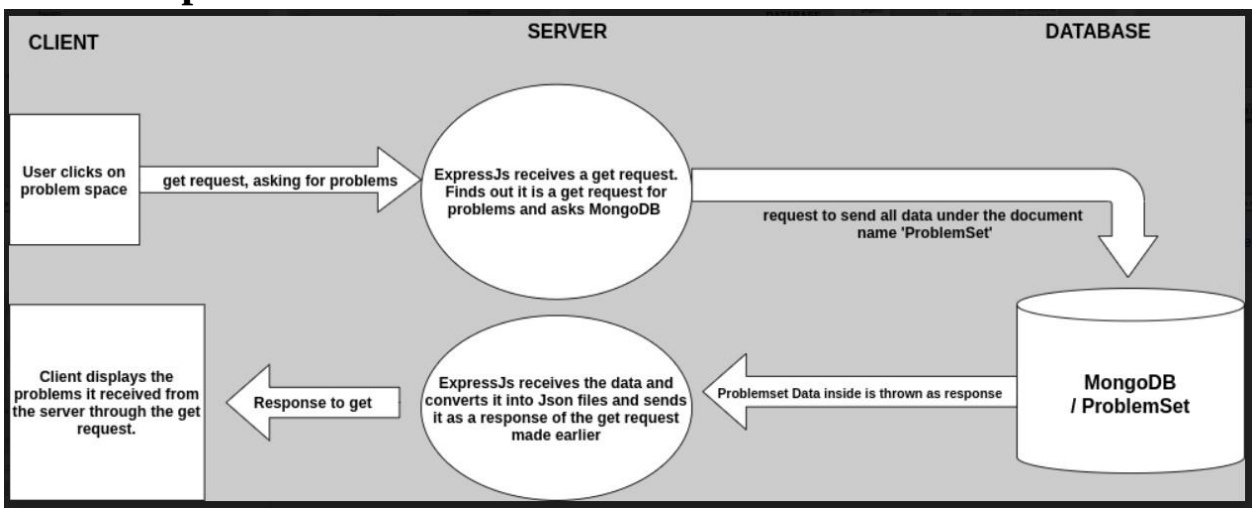

Figure 6.

Contest Space

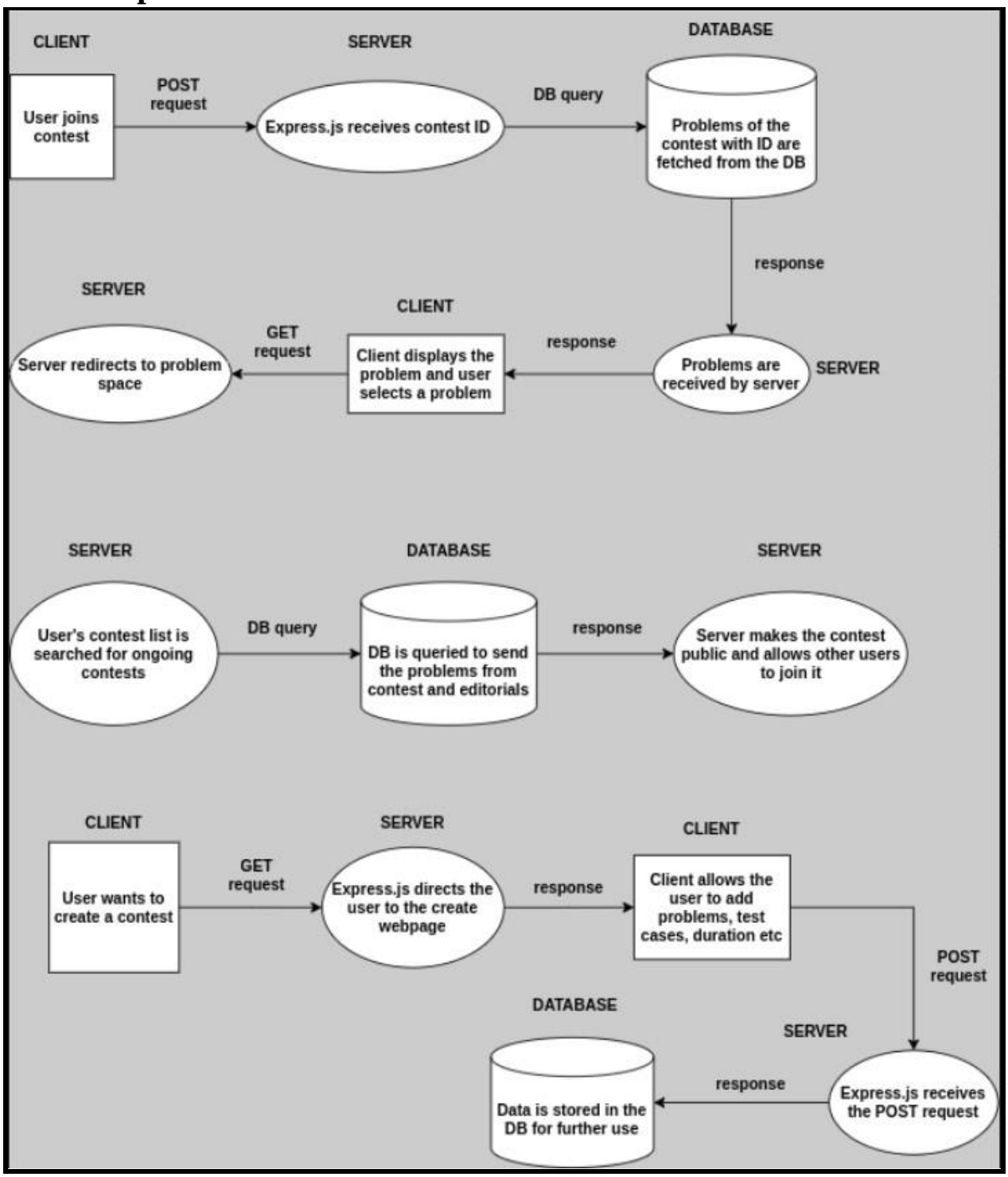

Figure 7.

Personal Development Space

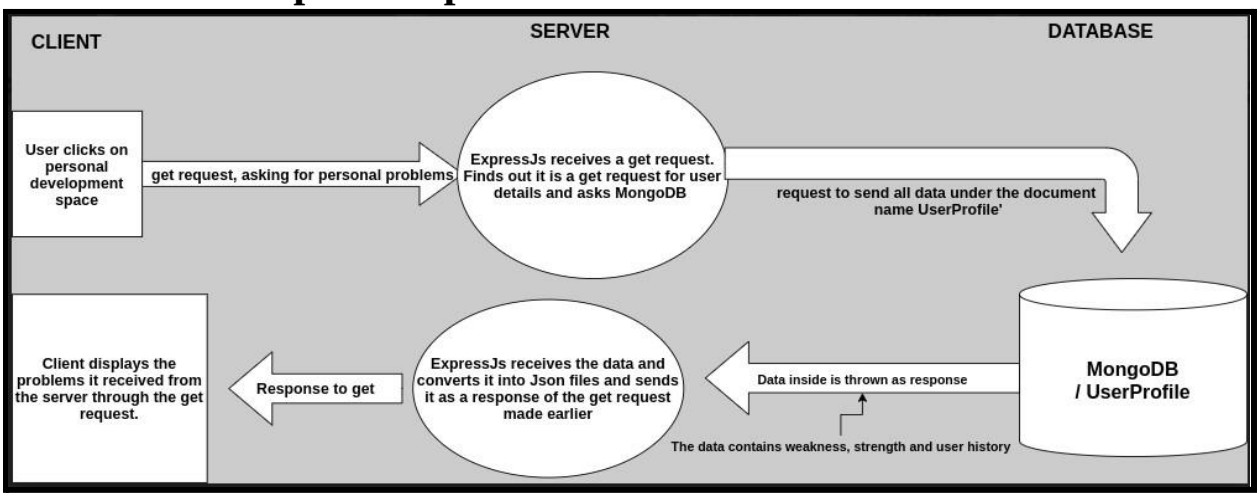

Figure 8.

User Space

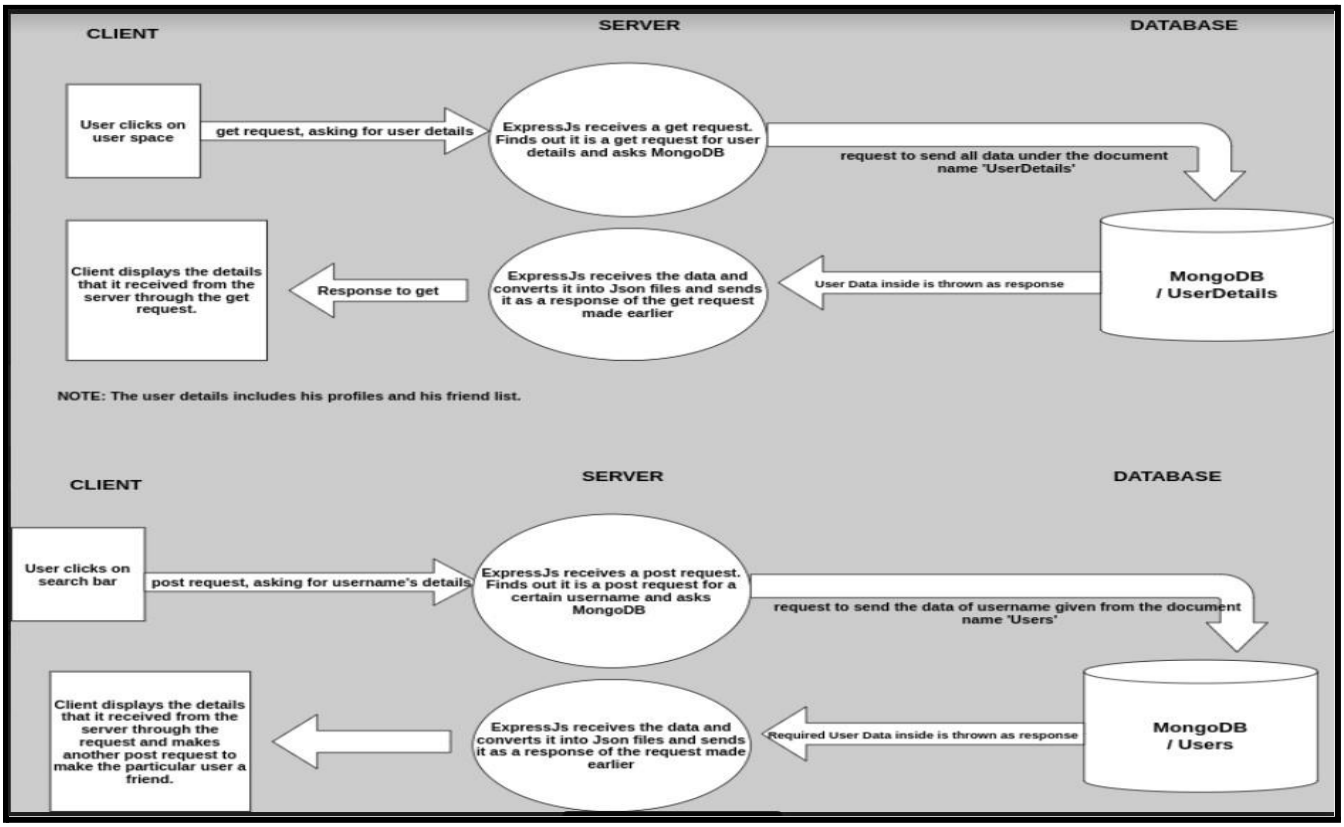

Figure 9.

Playlist space

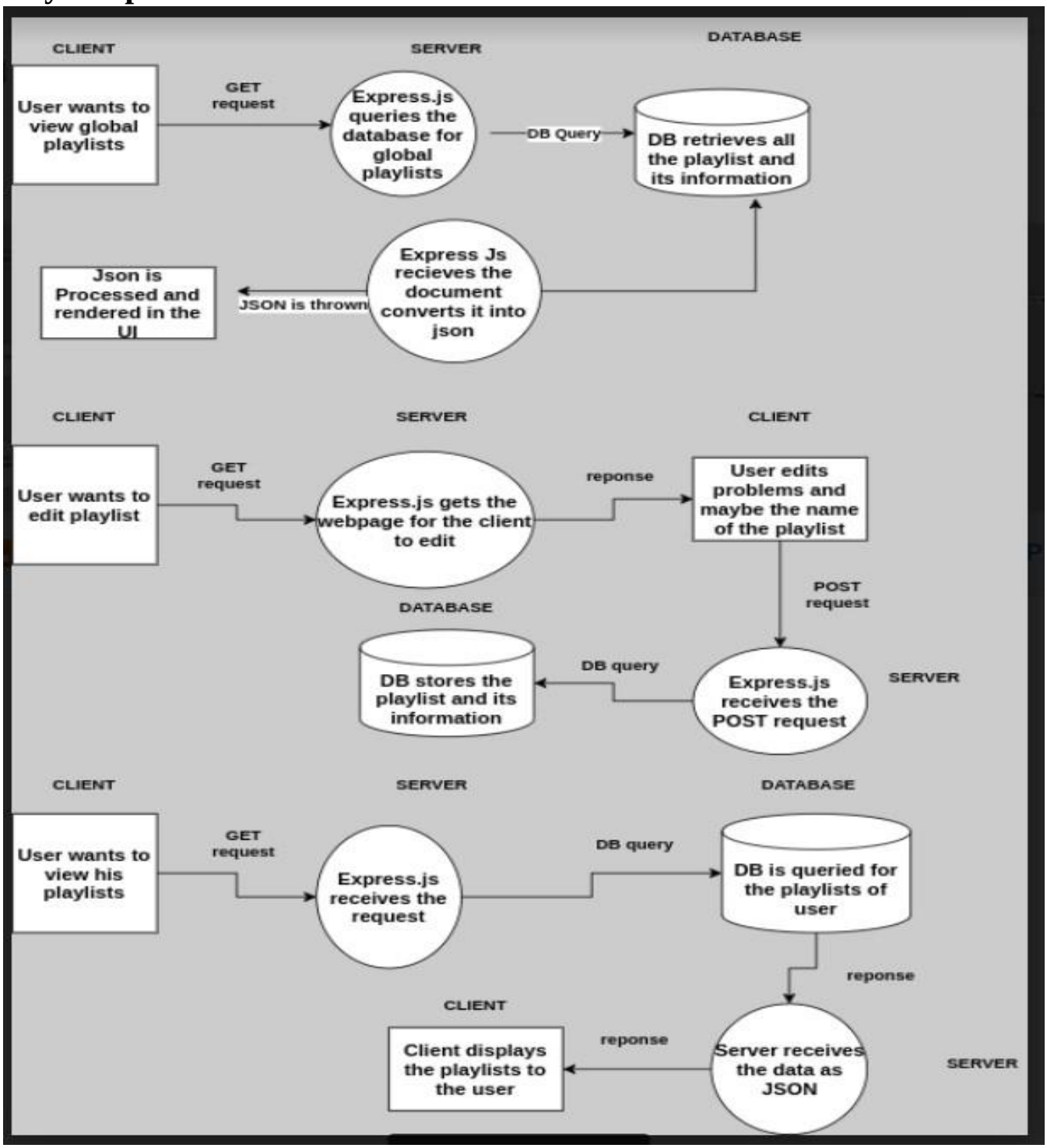

Figure 10.

System Architecture

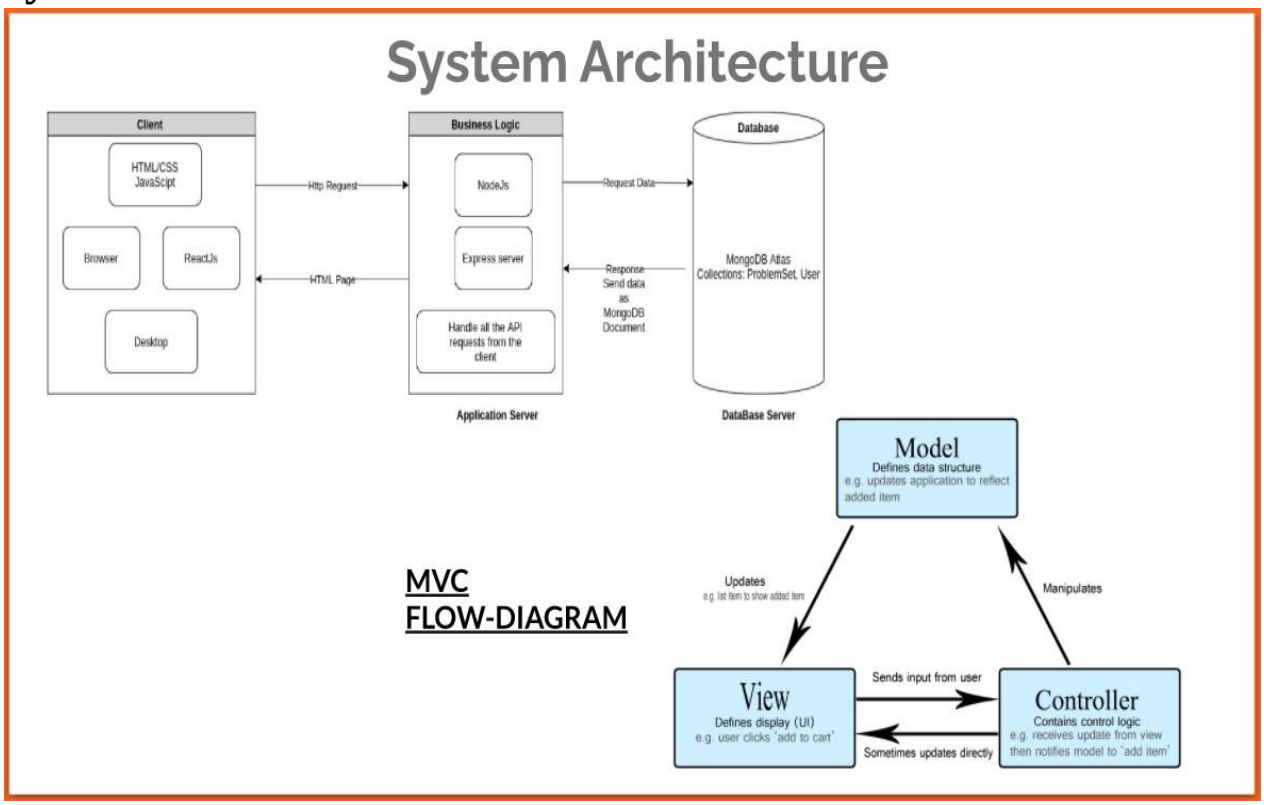

Figure 11.

Screenshots

Below are screenshots of the implementation of our project:

Authentication

Sign in

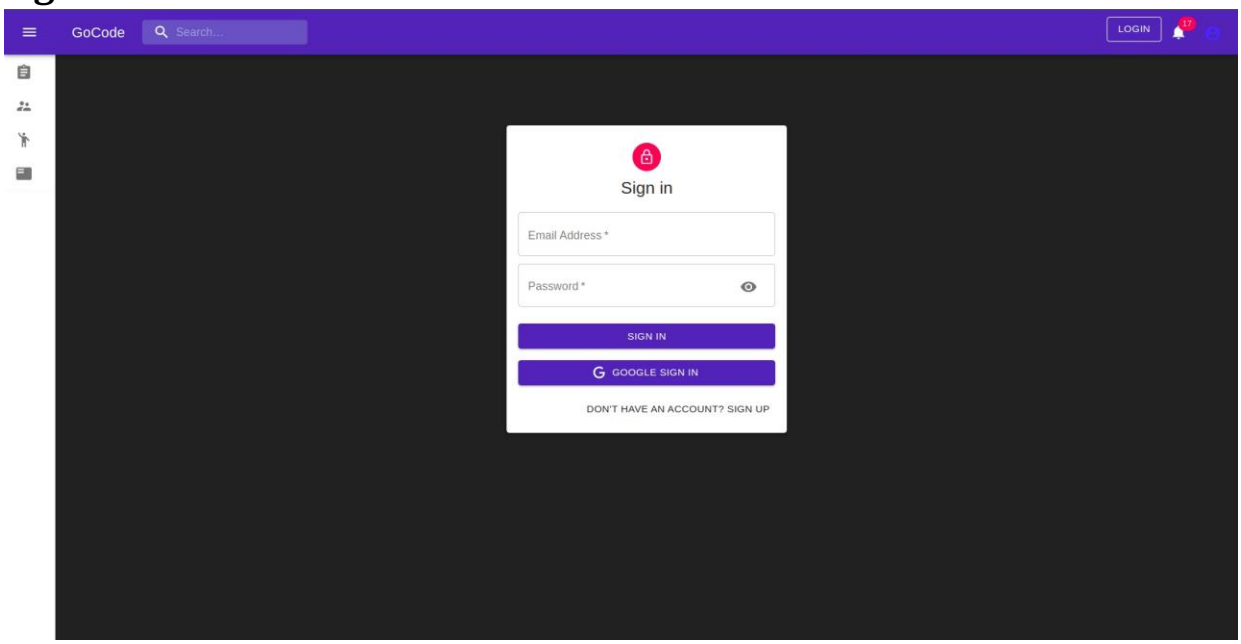

Figure 12.

Sign up

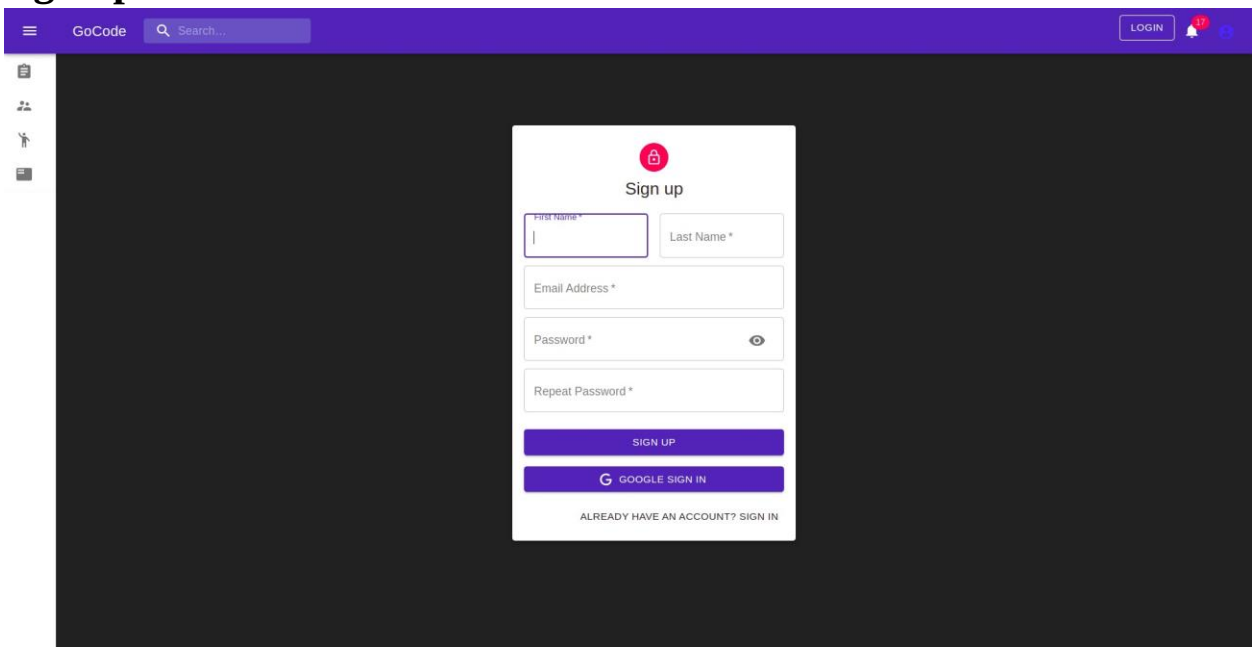

Figure 13.

Problem Space

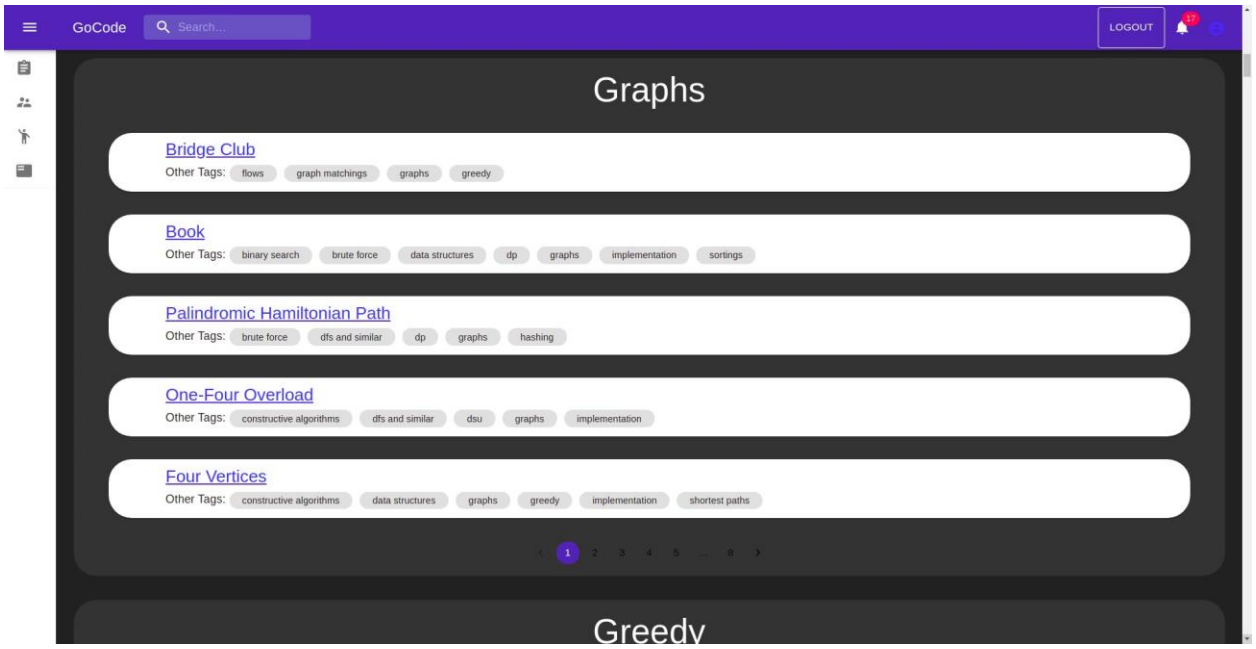

Figure 14.

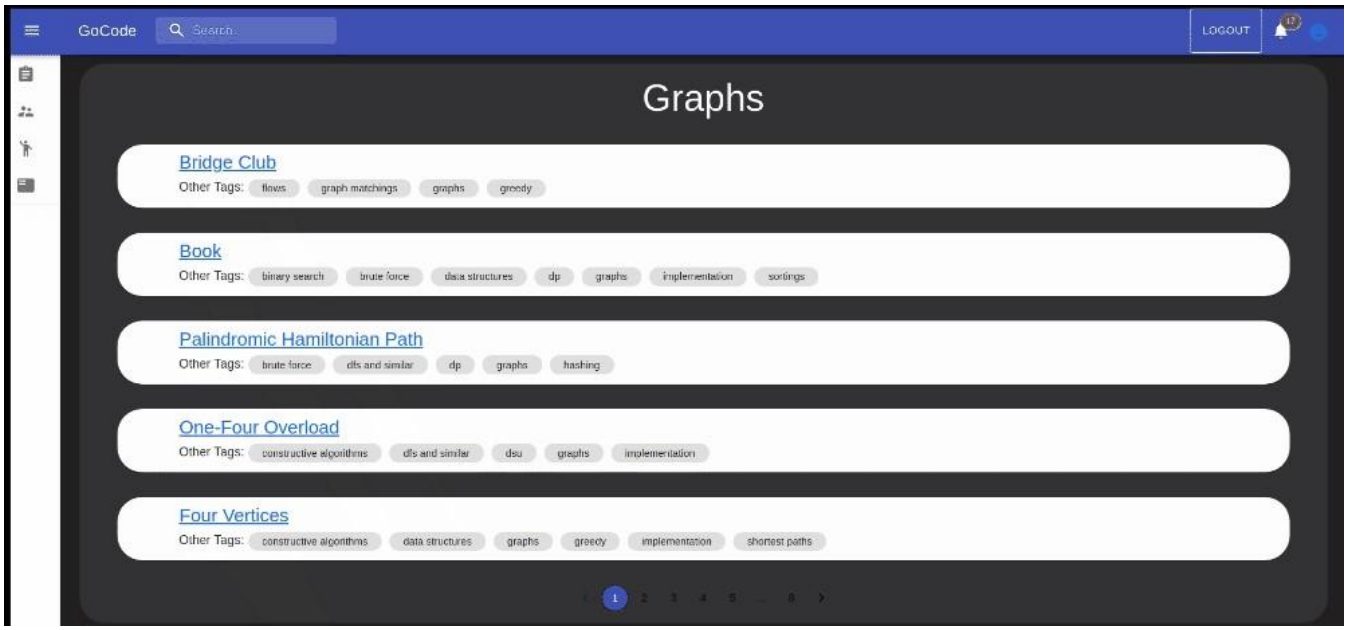

Figure 15.

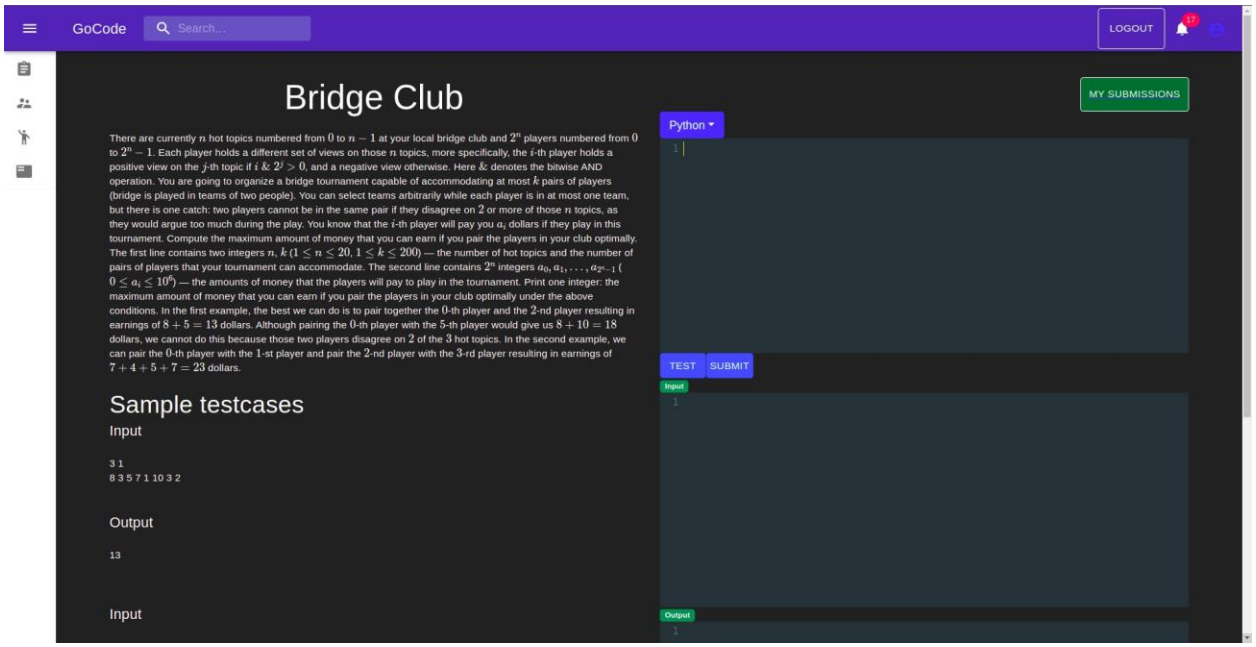

Figure 16.

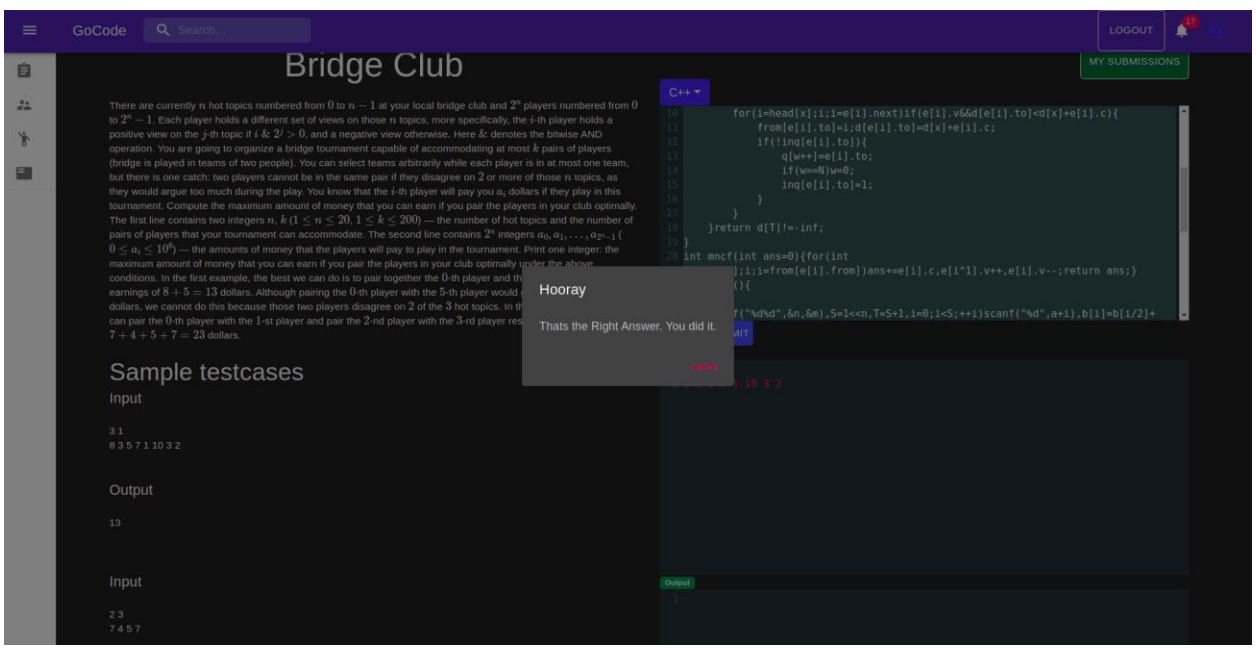

Figure 17.

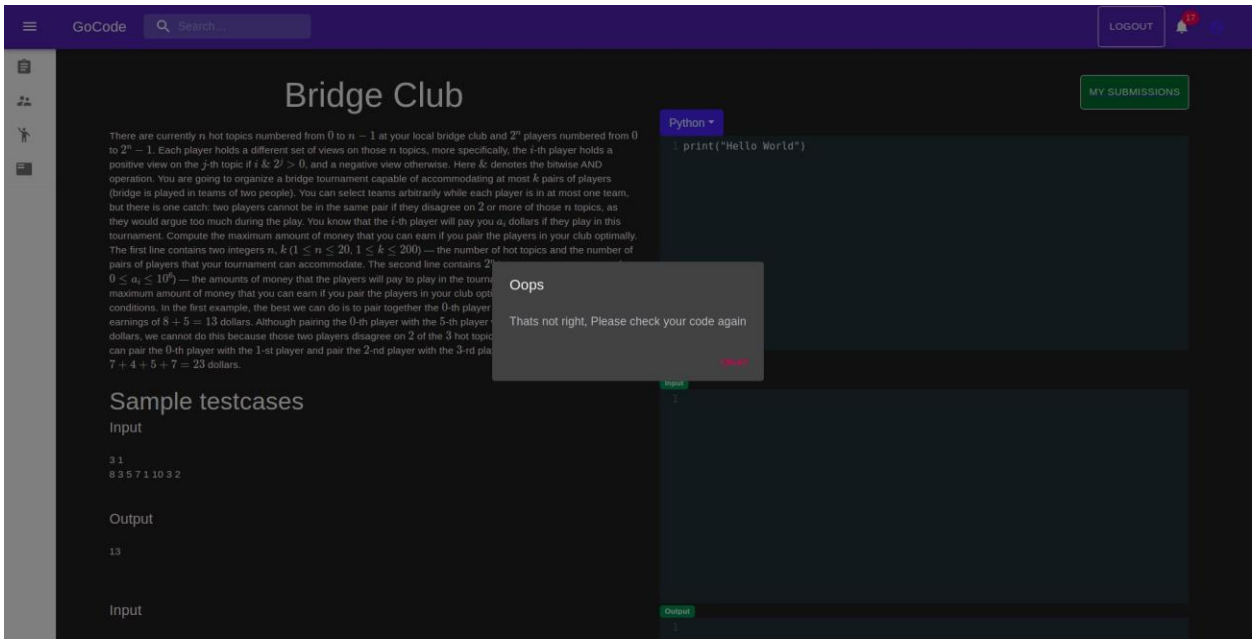

Figure 18.

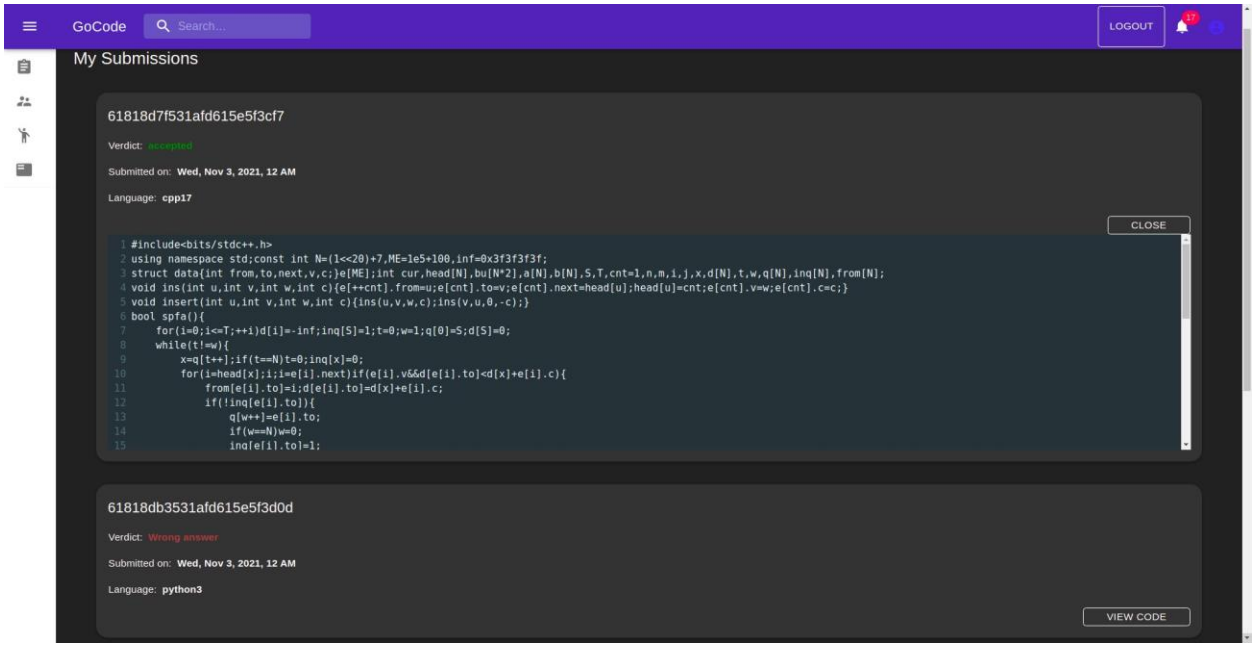

Figure 19.

Contest Space

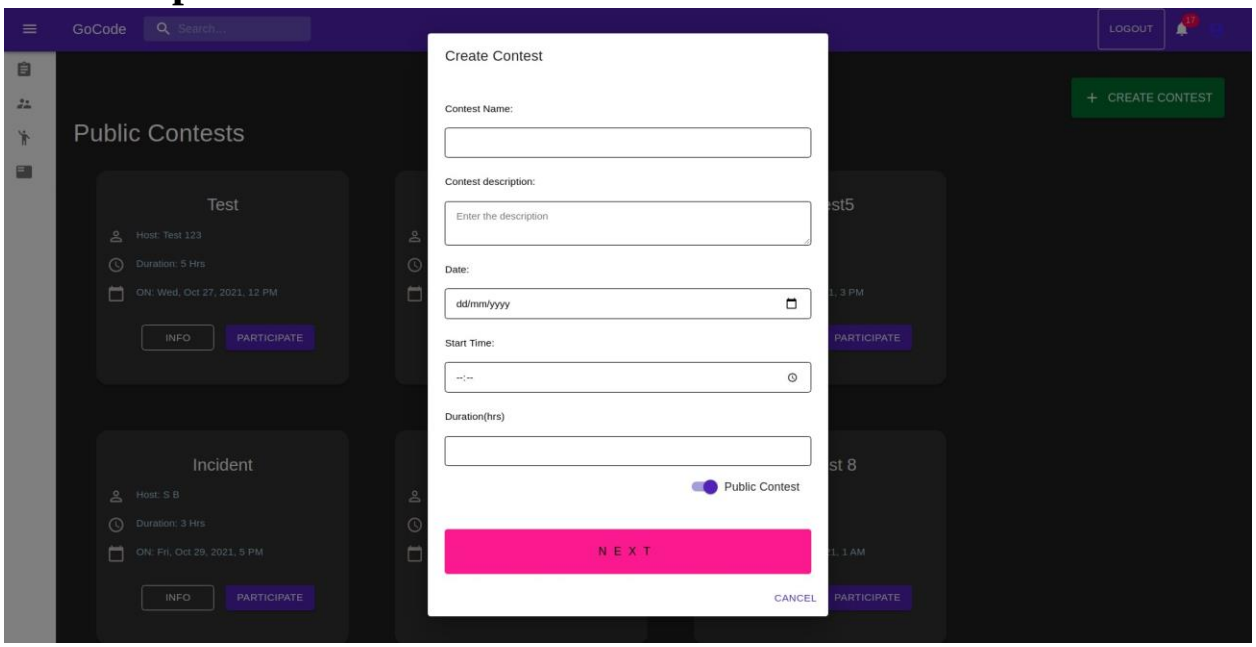

Figure 20.

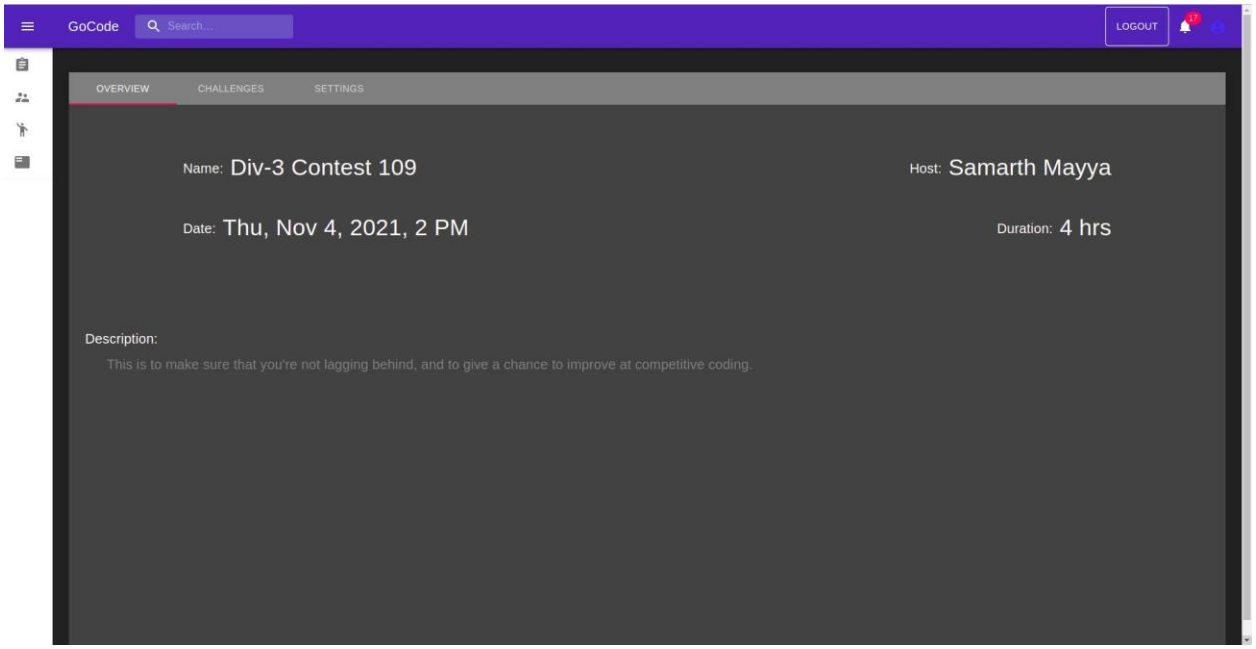

Figure 21.

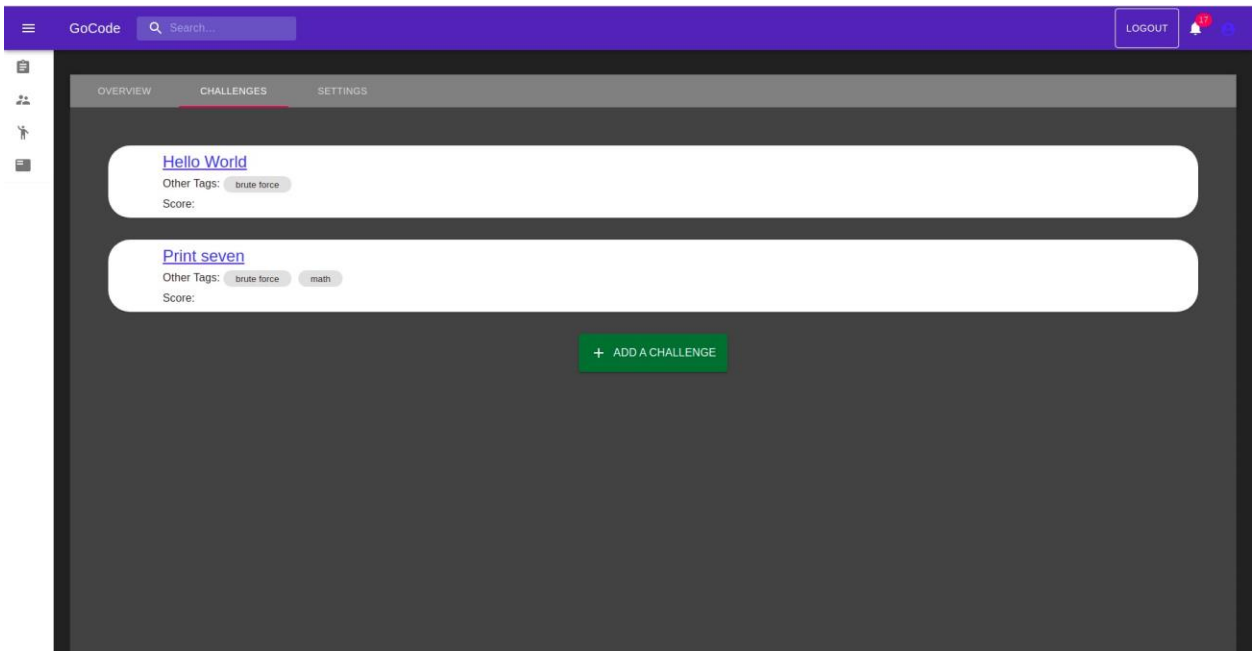

Figure 22.

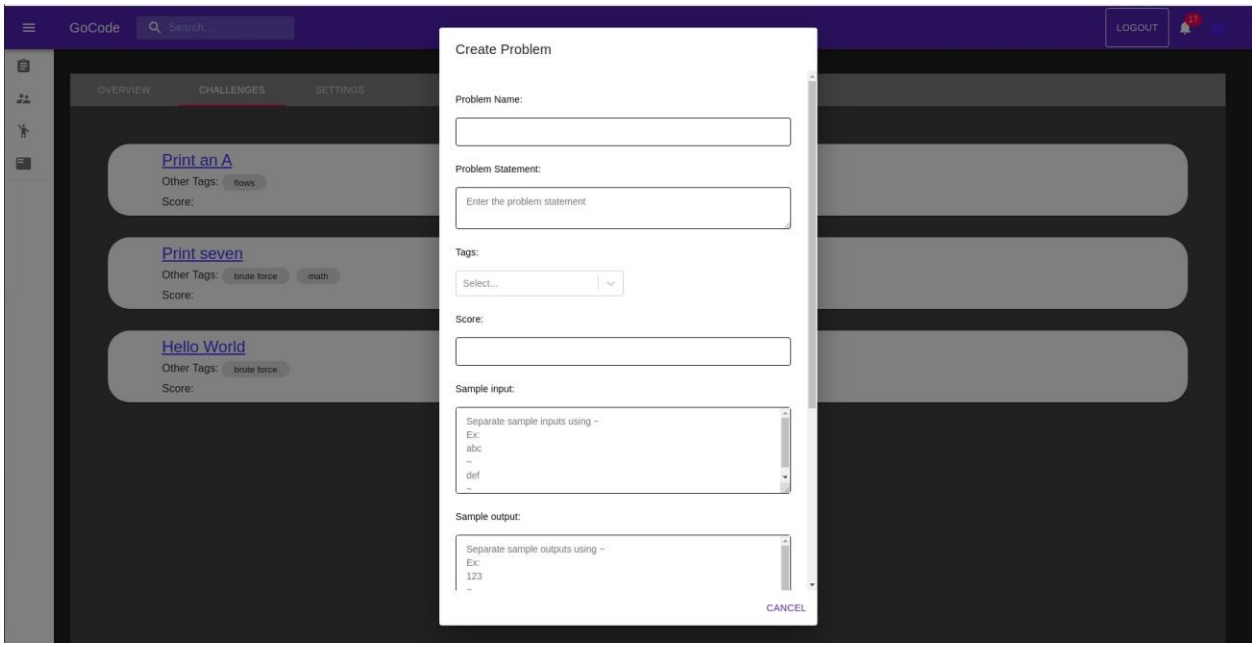

Figure 23.

Playlist Space

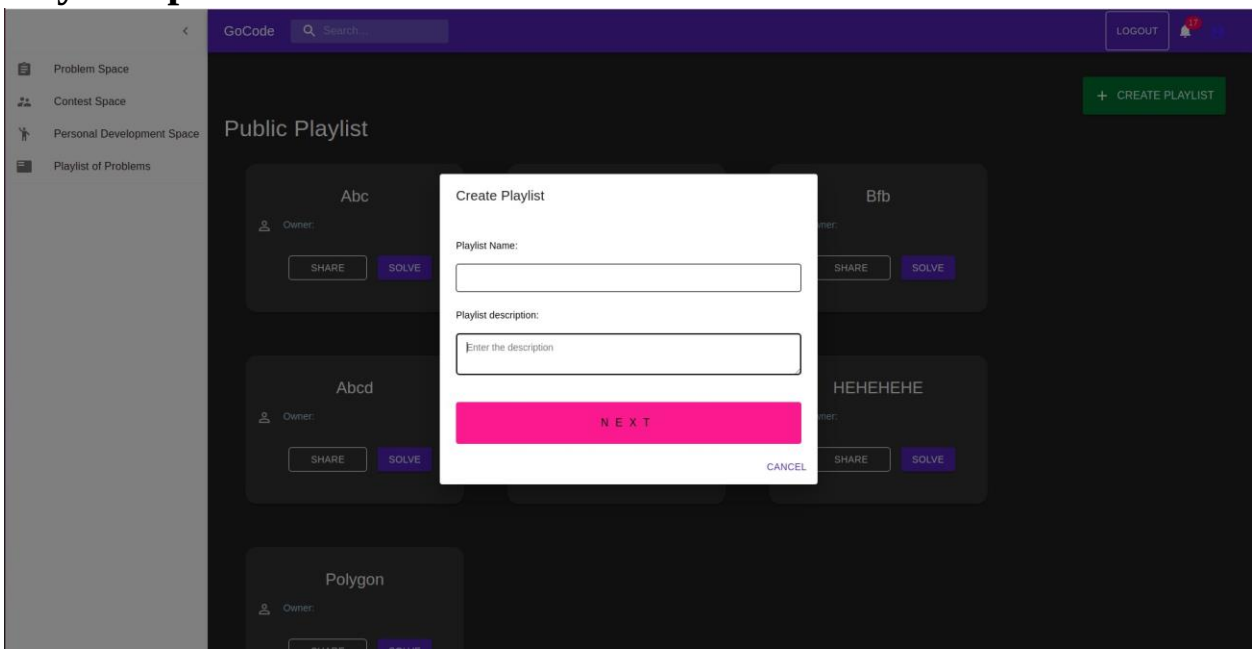

Figure 24.

Personal Development Space

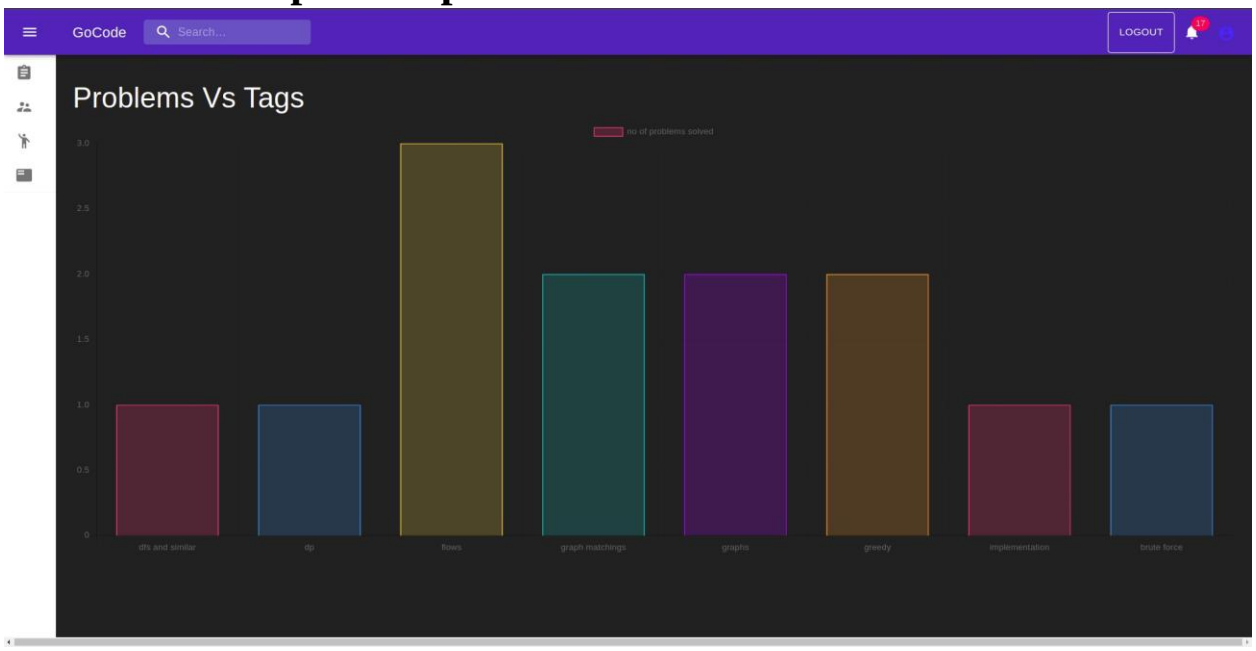

Figure 25.

Profile

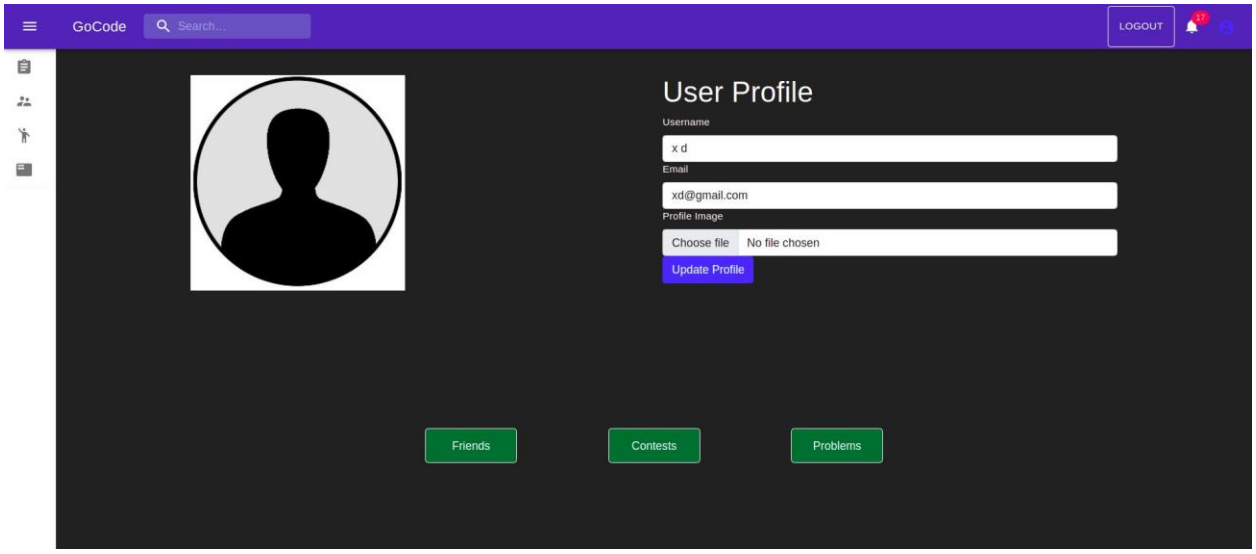

Figure 26.

Testing and Maintenance

Testing

Bugs and fixes:

- **Error**: When there was no internet connection, the circular bar was rotating,
  Solution: now there is an additional check for internet connectivity, and if the check fails, we display an error message.

- **Error**: Infinite number of GET requests were made in the contest-space which led to performance issues.
  Solution: Now the fetch method is accompanied by an initial check so that the method is run only once.

- **Issue**: every time fetching the problems was a performance overhead, so we had to come up with something efficient

Solution: Local storage feature offered in the browser developer tools, is now used to cache the problems to provide faster access.

- **Error**: Initially when the problems for the contest were added, we used to store it in a separate collection called ‘contest problems but moving forward with the project we encountered difficulties in CRUD operations
  Solution: Instead of having a new collection, we have added the IDs of the added problems into the contest data model.

- **Error**: Initially problem schema did not have the attribute of ‘score’, but contests require a score to rank the participants
  Solution: We added the score attribute for the problems.

- **Error**: The sidebar was collapsing by default and the user could not see what each icon meant.
  Solution: Hover on the text was added to each icon to give readability.

- **Error**: When the host adds a problem to the contest, the UI was not updated with the added problem. The host had to reload to see the added problem.
  Solution: Asynchronous functions were utilized to rectify the problem in the file “AddContest.js”.

- **Error**: Users were able to create contests for a past date.
  Solution: We are disabling the past dates in the form so that they cannot be selected.

- **Error**: Users were able to submit the empty forms.
  Solution: Form validation was added to make certain fields necessary to fill.

- **Error**: Some protected routes which should be only accessible by a few people, were accessible by everyone.
  Solution: We solved it by adding authentication middleware.

- **Error**: On refreshing the problem page, the code written by the user was lost.
  Solution: Solved it by storing the code in the local storage.

Unit tests

Unit tests are written to test the individual spaces using Mocha and Chai frameworks

Maintenance

- We are planning to verify emails before creating accounts for users

- As more users get added, we would require monetary assistance to continue the website as the data storage has costs attached to it.
- MongoDB is the database used and it offers invulnerability to cyberattacks and security threats.
- All the big websites today make use of MongoDB, so there is no worry when it comes to database corruption
- We are not displaying any information related to the private keys of the database/URLs of the DB in the front-end of our application, which provides another layer of protection.
- We are using JWT based authentication which is safer than cookie-session-based authentication.
- In the long run, the packages we use will have their own upgrades and once most of the packages get their upgrades, we will have to upgrade them to the latest edition and alter the code to avoid depreciation.
- The technologies we use will also have their own updates at some point which will imply updates in our project and make syntactic changes to keep up with the latest versions.

Conclusion and Future Treads

We have successfully built the coding platform named GoCode which helps the students prepare for various coding tests and technical interview rounds in their internship or placement season.

There are quite a good number of future improvements which we intend to make in our project, like providing the user with even more problems, adding a timer for every problem the user starts, providing much more compile language options, adding different themes to the code editor. We aim to keep improving our website to the fullest benefits of the end user.